

Hindi Vidya Prachar Samiti's
RAMNIRANJAN JHUNJHUNWALA COLLEGE OF
ARTS, SCIENCE AND COMMERCE
(EMPOWERED AUTONOMOUS)

Natural Language Processing



Name : Arulselvam Jegan

Roll no : 10443

Class : MSc. Data Science and Artificial Intelligence Part II



Ramniranjan Jhunjhunwala College of Arts, Science and Commerce

Department of Data Science and Artificial Intelligence

CERTIFICATE

This is to certify Arulselvam Jegan of MSc. Data Science and Artificial Intelligence Roll No 10443 has successfully completed the practical of Natural Language Processing during the Academic Year 2025-2026.

Date :

**(Prof. Shweta Gupta)
Prof-In-Charge**

External Examiner

INDEX

Sr. No.	Practicals	Date	Sign
1	Web Scraping: Scrape data from a web page & store it into csv format		
2	Implementation of Sentimental Analysis		
3	Implementation of Text Preprocessing		
4	Demonstrate the Parser in NLP		
5	Perform Feature Extraction technique in NLP task		
6	One Hot Encoding		
7	Bag-Of-Words(BOW)		
8	N-Grams		
9	Term Frequency Inverse Document Frequency		
10	Demonstrate the working of Word Embedding in Natural Language Processing		

Practical 1: Web Scraping: Scrape data from a web page & store it into csv format

```
[1] import pandas as pd
    from bs4 import BeautifulSoup
    import requests

[2] url = "https://www.scrapethissite.com/pages/simple/"
    response = requests.get(url)
    soup = BeautifulSoup(response.text, "html.parser")
    countries = soup.find_all("div", class_="country")

[3] data = []
    for country in countries:
        name = country.find("h3", class_="country-name").get_text(strip=True)
        capital = country.find("span", class_="country-capital").get_text(strip=True)
        population = country.find("span", class_="country-population").get_text(strip=True)
        area = country.find("span", class_="country-area").get_text(strip=True)

        data.append([name, capital, population, area])

    df = pd.DataFrame(data, columns=["Country", "Capital", "Population", "Area"])
```

 df.head()

	Country	Capital	Population	Area
0	Andorra	Andorra la Vella	84000	468.0
1	United Arab Emirates	Abu Dhabi	4975593	82880.0
2	Afghanistan	Kabul	29121286	647500.0
3	Antigua and Barbuda	St. John's	86754	443.0
4	Anguilla	The Valley	13254	102.0

Next steps: [Generate code with df](#) [View recommended plots](#) [New interactive sheet](#)

```
[5] df.to_csv('output.csv', index=False)
```

output.csv X

1 to 10 of 250 entries Filter 			
Country	Capital	Population	Area
Andorra	Andorra la Vella	84000	468.0
United Arab Emirates	Abu Dhabi	4975593	82880.0
Afghanistan	Kabul	29121286	647500.0
Antigua and Barbuda	St. John's	86754	443.0
Anguilla	The Valley	13254	102.0
Albania	Tirana	2986952	28748.0
Armenia	Yerevan	2968000	29800.0
Angola	Luanda	13068161	1246700.0
Antarctica	None	0	1.4E7
Argentina	Buenos Aires	41343201	2766890.0

Show 10 per page

1 2 10 20 25

Practical 2: Implementation of Sentimental Analysis



```
from textblob import TextBlob

text = "That went better than I expected."

# Create a TextBlob object
blob = TextBlob(text)

# Get the sentiment polarity (-1 to 1)
polarity = blob.sentiment.polarity

# Get the sentiment subjectivity (0 to 1)
subjectivity = blob.sentiment.subjectivity

print(f"Text: {text}")
print(f"Polarity: {polarity}")
print(f"Subjectivity: {subjectivity}")

if polarity > 0:
    print("Overall Sentiment: Positive 😊")
elif polarity < 0:
    print("Overall Sentiment: Negative 😞")
else:
    print("Overall Sentiment: Neutral 😐")
```



```
Text: That went better than I expected.
Polarity: 0.2
Subjectivity: 0.45
Overall Sentiment: Positive 😊
```

Practical 3: Data Implementation of Text Preprocessing

```
import re
import nltk
from nltk.corpus import stopwords
from nltk.stem import PorterStemmer

# Download stopwords (only first time)
nltk.download('stopwords')

# Sample text
text = "I am learning NLP. NLP is fun and very useful!"

print("Original Text:", text)

# 1. Lowercase
text = text.lower()
print("\nLowercase:", text)

# 2. Remove punctuation
text = re.sub(r'[^\w\s]', '', text)
print("\nNo punctuation:", text)

# 3. Tokenization (split into words)
words = text.split()
print("\nTokens:", words)

# 4. Remove Stopwords
stop_words = set(stopwords.words('english'))
filtered = [w for w in words if w not in stop_words]
print("\nWithout Stopwords:", filtered)

# 5. Stemming
ps = PorterStemmer()
stemmed = [ps.stem(w) for w in filtered]
print("\nAfter Stemming:", stemmed)
```

⇒ Original Text: I am learning NLP. NLP is fun and very useful!

Lowercase: i am learning nlp. nlp is fun and very useful!

No punctuation: i am learning nlp nlp is fun and very useful

Tokens: ['i', 'am', 'learning', 'nlp', 'nlp', 'is', 'fun', 'and', 'very', 'useful']

Without Stopwords: ['learning', 'nlp', 'nlp', 'fun', 'useful']

After Stemming: ['learn', 'nlp', 'nlp', 'fun', 'use']

[nltk_data] Downloading package stopwords to /root/nltk_data...

[nltk_data] Package stopwords is already up-to-date!

Practical 4: Demonstrate the Parser in NLP

```
33s !pip install spacy
!python -m spacy download en_core_web_lg
```

Requirement already satisfied: spacy in /usr/local/lib/python3.12/dist-packages (3.8.7)
Requirement already satisfied: spacy-legacy<3.1.0,>=3.0.11 in /usr/local/lib/python3.12/dist-packages (from spacy) (3.0.12)

```
[9] import spacy
from rich.console import Console
from rich.table import Table
```

```
nlp = spacy.load("en_core_web_sm")

sentence = "Cows graze under the tall green tree"
doc = nlp(sentence)

console = Console()
table = Table(title="Dependency Parse")

table.add_column("Token", style="cyan", justify="center")
table.add_column("Head", style="magenta", justify="center")
table.add_column("Dependency Tag", style="green", justify="center")

for token in doc:
    table.add_row(token.text, token.head.text, token.dep_)

console.print(table)
```

Dependency Parse

Token	Head	Dependency Tag
Cows	graze	compound
graze	graze	ROOT
under	graze	prep
the	tree	det
tall	tree	amod
green	tree	amod
tree	under	pobj

```
import spacy
from spacy import displacy

# Load English tokenizer, POS tagger, parser, NER
nlp = spacy.load("en_core_web_lg")

# Sample sentence
sentence = "Cows graze under the tall green tree"

# Process the sentence
doc = nlp(sentence)

# Print tokens and their dependency relations
print(f"{'Token':<15}{'Dependency':<20}{'Head'}")
print("-" * 50)
for token in doc:
    print(f"{token.text:<15}{token.dep_:<20}{token.head.text}")
```

Token	Dependency	Head
Cows	nsubj	graze
graze	ROOT	graze
under	prep	graze
the	det	tree
tall	amod	tree
green	amod	tree
tree	pobj	under

Practical 5: Perform Feature Extraction technique in NLP task

```
[ ] documents = [  
    "Natural Language Processing is amazing.",  
    "Machine learning models need features.",  
    "Feature extraction converts text into numbers.",  
    "Text data needs preprocessing and vectorization.",  
    "Word embeddings provide semantic meaning."  
]
```

```
# 1. Bag of Words (CountVectorizer)  
from sklearn.feature_extraction.text import CountVectorizer  
  
vectorizer = CountVectorizer()  
X_bow = vectorizer.fit_transform(documents)  
  
print("\n◆ Bag of Words (BoW) Vocabulary:\n", vectorizer.get_feature_names_out())  
print("\n◆ BoW Matrix:\n", X_bow.toarray())
```



◆ Bag of Words (BoW) Vocabulary:
['amazing' 'and' 'converts' 'data' 'embeddings' 'extraction' 'feature'
'features' 'into' 'is' 'language' 'learning' 'machine' 'meaning' 'models'
'natural' 'need' 'needs' 'numbers' 'preprocessing' 'processing' 'provide'
'semantic' 'text' 'vectorization' 'word']

◆ BoW Matrix:
[[1 0 0 0 0 0 0 0 1 1 0 0 0 0 1 0 0 0 0 1 0 0 0 0 0]
[0 0 0 0 0 0 0 1 0 0 0 1 1 0 1 0 1 0 0 0 0 0 0 0 0]
[0 0 1 0 0 1 1 0 1 0 0 0 0 0 0 0 0 0 0 1 0 0 0 0 1 0 0]
[0 1 0 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 1 0 1 0 0 0 1 1 0]
[0 0 0 0 1 0 0 0 0 0 0 0 0 1 0 0 0 0 0 0 0 1 1 0 0 1]]



```
# 2. TF-IDF (TfidfVectorizer)  
from sklearn.feature_extraction.text import TfidfVectorizer  
  
tfidf = TfidfVectorizer()  
X_tfidf = tfidf.fit_transform(documents)  
  
print("\n◆ TF-IDF Vocabulary:\n", tfidf.get_feature_names_out())  
print("\n◆ TF-IDF Matrix:\n", X_tfidf.toarray())
```



◆ TF-IDF Vocabulary:

```
['amazing' 'and' 'converts' 'data' 'embeddings' 'extraction' 'feature'
'features' 'into' 'is' 'language' 'learning' 'machine' 'meaning' 'models'
'natural' 'need' 'needs' 'numbers' 'preprocessing' 'processing' 'provide'
'semantic' 'text' 'vectorization' 'word']
```

◆ TF-IDF Matrix:

```
[[0.4472136 0.          0.          0.          0.          0.
 0.          0.          0.          0.4472136 0.4472136 0.
 0.          0.          0.          0.4472136 0.          0.
 0.          0.          0.4472136 0.          0.          0.
 0.          0.          ]
 [0.          0.          0.          0.          0.          0.
 0.          0.4472136 0.          0.          0.          0.4472136
 0.4472136 0.          0.4472136 0.          0.4472136 0.
 0.          0.          0.          0.          0.          0.
 0.          0.          ]
 [0.          0.          0.42066906 0.          0.          0.42066906
 0.42066906 0.          0.42066906 0.          0.          0.
 0.          0.          0.          0.          0.          0.
 0.42066906 0.          0.          0.          0.          0.33939315
 0.          0.          ]
 [0.          0.42066906 0.          0.42066906 0.          0.
 0.          0.          0.          0.          0.          0.
 0.          0.          0.          0.          0.          0.42066906
 0.          0.42066906 0.          0.          0.          0.33939315
 0.42066906 0.          ]
 [0.          0.          0.          0.          0.4472136 0.
 0.          0.          0.          0.          0.          0.
 0.          0.4472136 0.          0.          0.          0.
 0.          0.          0.          0.4472136 0.4472136 0.
 0.          0.4472136 ]]
```



3. Word Embeddings (spaCy vectors)

```
import spacy

nlp = spacy.load("en_core_web_lg")

vectors = []
for doc in documents:
    doc_vec = nlp(doc).vector
    vectors.append(doc_vec)

print("\n◆ Word Embeddings Shape (1st doc):", vectors[0].shape)
print("◆ First 5 values of Doc 1 vector:\n", vectors[0][:5])
```



◆ Word Embeddings Shape (1st doc): (300,)

◆ First 5 values of Doc 1 vector:

```
[-0.24087083  0.209805  -0.00093312 -0.11442658  0.09268666]
```

Practical 6: One Hot Encoding

```
[12] from sklearn.preprocessing import OneHotEncoder
import numpy as np

sentences = [
    "I enjoy machine learning.",
    "AI is powerful.",
    "I love deep learning."
]
```

```
[13] # Tokenize into words
words = [word for sentence in sentences for word in sentence.split()]

# Unique vocabulary
vocab = sorted(set(words))

# Reshape for encoder
vocab_array = np.array(vocab).reshape(-1, 1)

# Initialize encoder (use sparse_output instead of sparse)
encoder = OneHotEncoder(sparse_output=False)

# Fit and transform
one_hot_vectors = encoder.fit_transform(vocab_array)
```



```
max_len = max(len(word) for word in vocab)
print("One-Hot Encoded Vocabulary:\n")
for word, vector in zip(vocab, one_hot_vectors):
    print(f"{word:<{max_len}} --> {vector.astype(int)}")
```



One-Hot Encoded Vocabulary:

```
AI          --> [1 0 0 0 0 0 0 0 0]
I           --> [0 1 0 0 0 0 0 0 0]
deep        --> [0 0 1 0 0 0 0 0 0]
enjoy       --> [0 0 0 1 0 0 0 0 0]
is          --> [0 0 0 0 1 0 0 0 0]
learning.   --> [0 0 0 0 0 1 0 0 0]
love        --> [0 0 0 0 0 0 1 0 0]
machine     --> [0 0 0 0 0 0 0 1 0]
powerful.   --> [0 0 0 0 0 0 0 0 1]
```

Practical 7: Bag-Of-Words(BOW)

```
[ ] import nltk
    from nltk.corpus import stopwords
    from nltk.tokenize import word_tokenize
    import string
    import pandas as pd

    nltk.download('punkt_tab')
```

```
⇒ [nltk_data] Downloading package punkt_tab to /root/nltk_data...
   [nltk_data]   Package punkt_tab is already up-to-date!
   True
```

```
[ ] corpus = [
    "Python is amazing and fun.",
    "Python is not just fun but also powerful.",
    "Learning Python is fun!",
]
```

```
[ ] def preprocess(text):
    # Lowercase
    text = text.lower()
    # Remove punctuation
    text = text.translate(str.maketrans("", "", string.punctuation))
    # Tokenize
    tokens = word_tokenize(text)
    return tokens
```

```
[18] processed_corpus = [preprocess(sentence) for sentence in corpus]
```

```
# Build vocabulary (unique words across corpus)
vocabulary = sorted(set(word for sentence in processed_corpus for word in sentence))
print("Vocabulary:", vocabulary)
print("Vocabulary size:", len(vocabulary))
```

```
⇒ Vocabulary: ['also', 'amazing', 'and', 'but', 'fun', 'is', 'just', 'learning', 'not', 'powerful', 'python']
Vocabulary size: 11
```

```
[19] def create_bow_vector(sentence, vocab):
    vector = [0] * len(vocab)
    for word in sentence:
        if word in vocab:
            idx = vocab.index(word)
            vector[idx] += 1
    return vector

# Create vectors for all sentences
bow_vectors = [create_bow_vector(sentence, vocabulary) for sentence in processed_corpus]
```

```
[20] # Create BoW vector for each sentence in the processed corpus
bow_vectors = [create_bow_vector(sentence, vocabulary) for sentence in processed_corpus]
print("Bag of Words Vectors:")
for vector in bow_vectors:
    print(vector)
```

```
⇒ Bag of Words Vectors:
[0, 1, 1, 0, 1, 1, 0, 0, 0, 0, 1]
[1, 0, 0, 1, 1, 1, 1, 0, 1, 1, 1]
[0, 0, 0, 0, 1, 1, 0, 1, 0, 0, 1]
```

Practical 8: N-Grams

```
[21] import pandas as pd
import nltk
from nltk.util import ngrams
from nltk.corpus import reuters
from nltk import trigrams
from collections import defaultdict
nltk.download("punkt")
nltk.download("reuters")
```

⇒ [nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data] Unzipping tokenizers/punkt.zip.
[nltk_data] Downloading package reuters to /root/nltk_data...
True

```
[22] df = pd.DataFrame({
    "word": ["apple", "banana", "cherry", "date"],
    "POS": ["pronoun", "verb", "noun", "adverb"]
})
```

```
[23] one_hot = pd.get_dummies(df["word"], prefix="token")
print(one_hot)
```

⇒

	token_apple	token_banana	token_cherry	token_date
0	True	False	False	False
1	False	True	False	False
2	False	False	True	False
3	False	False	False	True

```
[24] def generate_ngrams(tokens, n):
    return [tokens[i:i+n] for i in range(len(tokens)-n+1)]
```

```
sentence = "I love Python programming".split()
print("\n=== Simple N-Grams ===")
print("Unigrams :", generate_ngrams(sentence, 1))
print("Bigrams  :", generate_ngrams(sentence, 2))
print("Trigrams  :", generate_ngrams(sentence, 3))
```

⇒

```
=== Simple N-Grams ===
Unigrams : [['I'], ['love'], ['Python'], ['programming']]
Bigrams  : [['I', 'love'], ['love', 'Python'], ['Python', 'programming']]
Trigrams : [['I', 'love', 'Python'], ['love', 'Python', 'programming']]
```

```
[25] words = [w.lower() for w in reuters.words() if w.isalpha()]
```

```
# Build trigrams
tri_grams = list(trigrams(words))

# Build trigram probability model
model = defaultdict(lambda: defaultdict(lambda: 0))
for w1, w2, w3 in tri_grams:
    model[(w1, w2)][w3] += 1

# Normalize counts to probabilities
for w1_w2 in model:
    total_count = float(sum(model[w1_w2].values()))
    for w3 in model[w1_w2]:
        model[w1_w2][w3] /= total_count
```

```
[26] def predict_next_word(w1, w2):
      next_word_probs = model[w1, w2]
      if next_word_probs:
          return max(next_word_probs, key=next_word_probs.get)
      return "No prediction available"

print("Reuters Trigram Prediction")
print("Next word after ['the', 'market'] →", predict_next_word("the", "market"))
```

➡ Reuters Trigram Prediction
Next word after ['the', 'market'] → is

```
[27] data = {
      "id": [1, 2, 3],
      "text": [
          "I love natural language processing",
          "Natural language processing is fun",
          "I love deep learning"
      ]
  }

df = pd.DataFrame(data)
```

```
[28] def create_ngrams(text, n):
      tokens = text.lower().split()
      return list(ngrams(tokens, n))

# Add bigrams and trigrams to DataFrame
df["bigrams"] = df["text"].apply(lambda x: create_ngrams(x, 2))
df["trigrams"] = df["text"].apply(lambda x: create_ngrams(x, 3))
```

▶ df[["text", "bigrams", "trigrams"]]

	text	bigrams	trigrams
0	I love natural language processing	[(i, love), (love, natural), (natural, languag...	[(i, love, natural), (love, natural, language)...
1	Natural language processing is fun	[(natural, language), (language, processing), ...	[(natural, language, processing), (language, p...
2	I love deep learning	[(i, love), (love, deep), (deep, learning)]	[(i, love, deep), (love, deep, learning)]

Practical 9: Term Frequency Inverse Document Frequency

```
[ ] from sklearn.feature_extraction.text import TfidfVectorizer
import pandas as pd
```

```
[ ] documents = [
    "The cat sat on the mat.",
    "The dog sat on the log.",
    "The cat chased the mouse.",
    "The dog barked at the cat.",
    "Cats and dogs are common pets."
]
```

```
[ ] # Initialize the TF-IDF Vectorizer
vectorizer = TfidfVectorizer(stop_words='english') # remove common English stopwords

# Fit and transform the documents
tfidf_matrix = vectorizer.fit_transform(documents)

# Convert to a pandas DataFrame for easy viewing
df_tfidf = pd.DataFrame(tfidf_matrix.toarray(), columns=vectorizer.get_feature_names_out())
```

```
▶ # Display the TF-IDF values
print("TF-IDF Matrix:\n")
print(df_tfidf)
```

TF-IDF Matrix:

	barked	cat	cats	chased	common	dog	dogs	log	\
0	0.000000	0.462208	0.0	0.000000	0.0	0.000000	0.0	0.000000	
1	0.000000	0.000000	0.0	0.000000	0.0	0.531772	0.0	0.659118	
2	0.000000	0.427993	0.0	0.63907	0.0	0.000000	0.0	0.000000	
3	0.690159	0.462208	0.0	0.000000	0.0	0.556816	0.0	0.000000	
4	0.000000	0.000000	0.5	0.000000	0.5	0.000000	0.5	0.000000	

	mat	mouse	pets	sat
0	0.690159	0.000000	0.0	0.556816
1	0.000000	0.000000	0.0	0.531772
2	0.000000	0.63907	0.0	0.000000
3	0.000000	0.000000	0.0	0.000000
4	0.000000	0.000000	0.5	0.000000

Practical 10: Demonstrate the working of Word Embedding in Natural Language Processing

```
%pip uninstall numpy scipy gensim -y
%pip install numpy==1.26.4 scipy==1.13.1 gensim
```

```
Found existing installation: numpy 1.26.4
Uninstalling numpy-1.26.4:
  Successfully uninstalled numpy-1.26.4
Found existing installation: scipy 1.13.1
Uninstalling scipy-1.13.1:
  Successfully uninstalled scipy-1.13.1
```

```
[2] from gensim.models import Word2Vec
    sentences = [
        ['the', 'cat', 'sat', 'on', 'the', 'mat'],
        ['the', 'dog', 'sat', 'on', 'the', 'rug'],
        ['the', 'cat', 'chased', 'the', 'mouse'],
        ['the', 'dog', 'barked', 'at', 'the', 'cat']
    ]
```

```
[3] model = Word2Vec(sentences, vector_size=50, window=3, min_count=1, sg=1)
```

```
[4] #1: Get the vector for a word
    vector_cat = model.wv['cat']
    print("Vector for 'cat':\n", vector_cat)
```

```
Vector for 'cat':
[-0.01631589  0.00899154 -0.00827435  0.00164885  0.01699699 -0.00892466
 0.00903494 -0.013574  -0.00709726  0.01879679 -0.00315515  0.00064249
-0.00828101 -0.01536522 -0.00301625  0.00494004 -0.00177567  0.01106763
-0.00548626  0.00452034  0.01091177  0.01669204 -0.00290721 -0.01841634
 0.00874107  0.00114392  0.01488411 -0.00162636 -0.00527696 -0.01750593
-0.00171294  0.00565275  0.01080254  0.01410523 -0.01140614  0.00371746
 0.01217764 -0.0095963  -0.00621408  0.01359538  0.00326288  0.00037996
 0.0069475  0.00043539  0.01923791  0.01012151 -0.01783448 -0.01408312
 0.00180324  0.01278482]
```

```
# 2: Find most similar words to 'cat'
similar_to_cat = model.wv.most_similar('cat', topn=7)
print("\nTop 3 words similar to 'cat':")
for word, score in similar_to_cat:
    print(f"{word}: {score:.4f}")
```

```
Top 3 words similar to 'cat':
sat: 0.1249
barked: 0.0740
the: 0.0424
at: 0.0183
dog: 0.0111
chased: 0.0014
mat: -0.1191
```